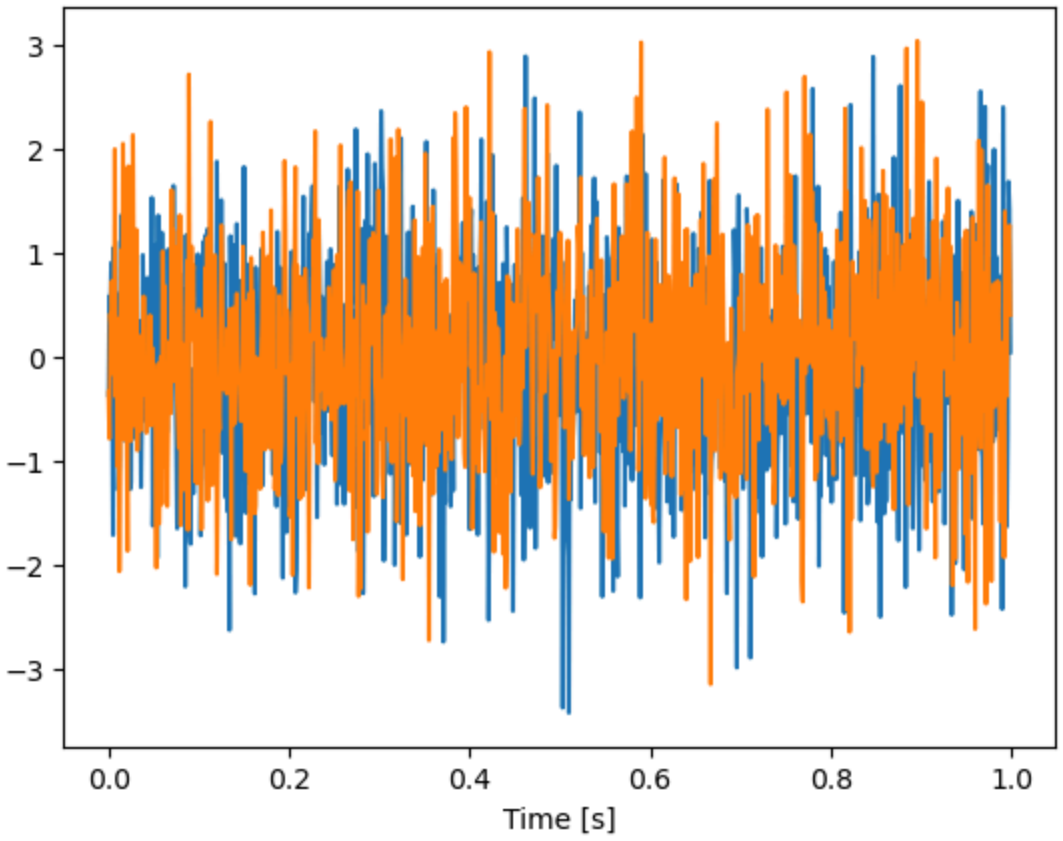
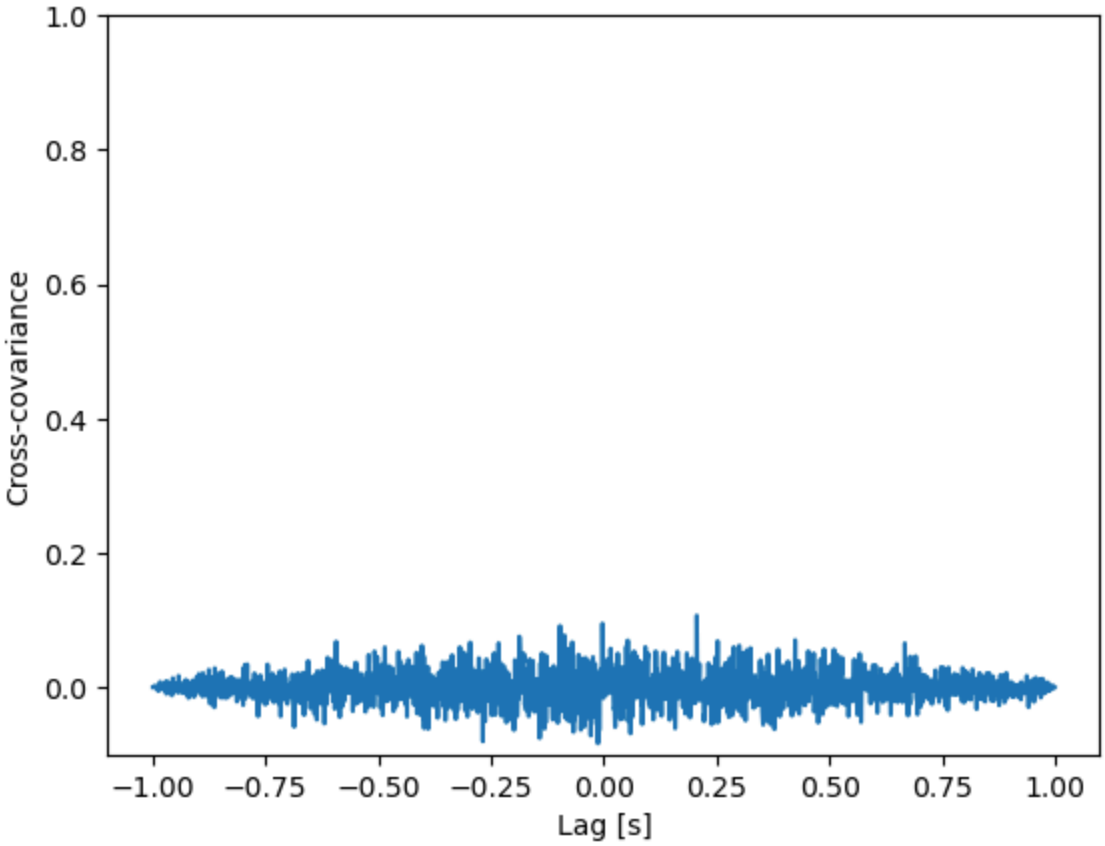
In [1]:
```
# Load modules we'll need.
from scipy.io import loadmat
import matplotlib.pyplot as plt
import numpy as np
from scipy.signal import spectrogram
```

In [2]:
```
N = 1000;
dt= 0.001;
T = N*dt;
x = np.random.randn(N)
y = np.random.randn(N)
t = np.arange(0,N)*dt

plt.plot(t,x)
plt.plot(t,y)
plt.xlabel('Time [s]');
```



In [3]:
```
cc_xy = 1/N * np.correlate(x-np.mean(x), y-np.mean(y), 'full')
lags = np.arange(-N + 1, N)                    # Create a lag axis,
plt.plot(lags * dt, cc_xy)                     # ... and plot the result.
plt.ylim([-0.1, 1])
plt.xlabel('Lag [s]')
plt.ylabel('Cross-covariance');
```



In [4]:
```
Xf = np.fft.fft(x - x.mean())           # Compute Fourier transform of x
Yf = np.fft.fft(y - y.mean())           # Compute Fourier transform of y

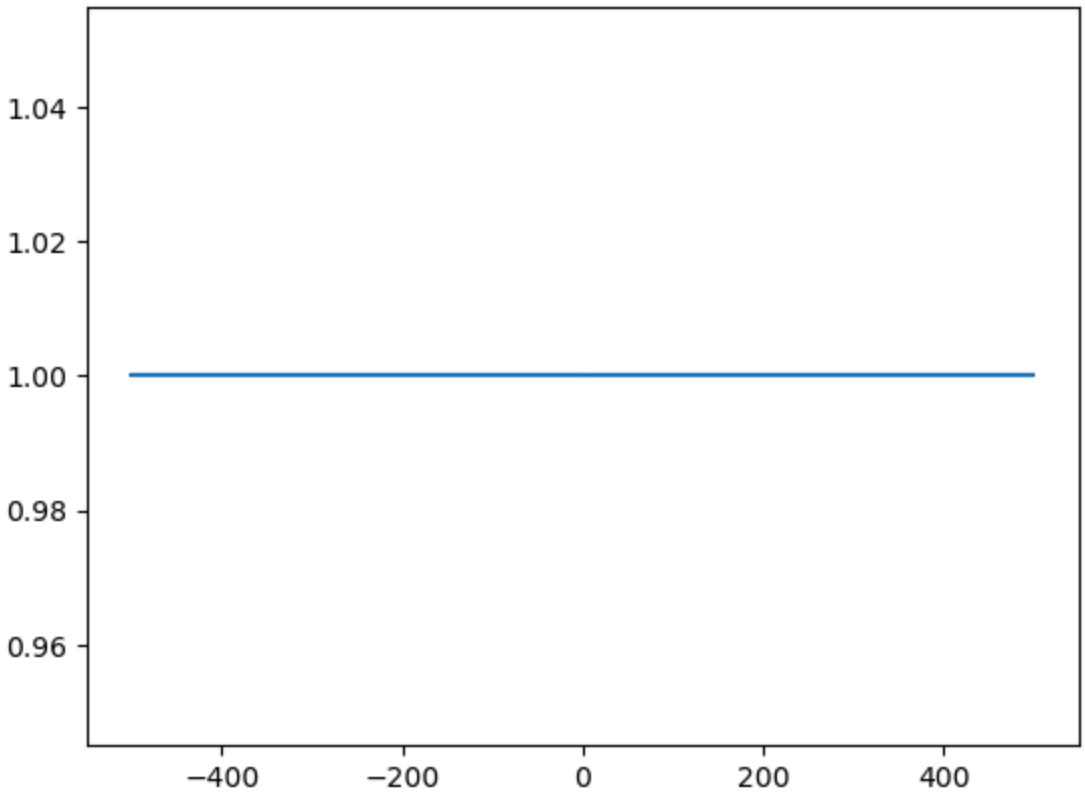
# Compute the spectra
Sxx = 2*dt**2/T * Xf * np.conj(Xf)  # Spectrum of x
Syy = 2*dt**2/T * Yf * np.conj(Yf)  # ... and y
Sxy = 2*dt**2/T * Xf * np.conj(Yf)  # ... and the cross spectrum

# Compute the coherence.
cohr = np.abs(Sxy) / (np.sqrt(Sxx) * np.sqrt(Syy))

# Define a frequency axis.
f = np.fft.fftfreq(N, dt)

# Plot the result.
plt.plot(f, cohr.real)
```

Out[4]:
```
[<matplotlib.lines.Line2D at 0x1b3b1b980>]
```

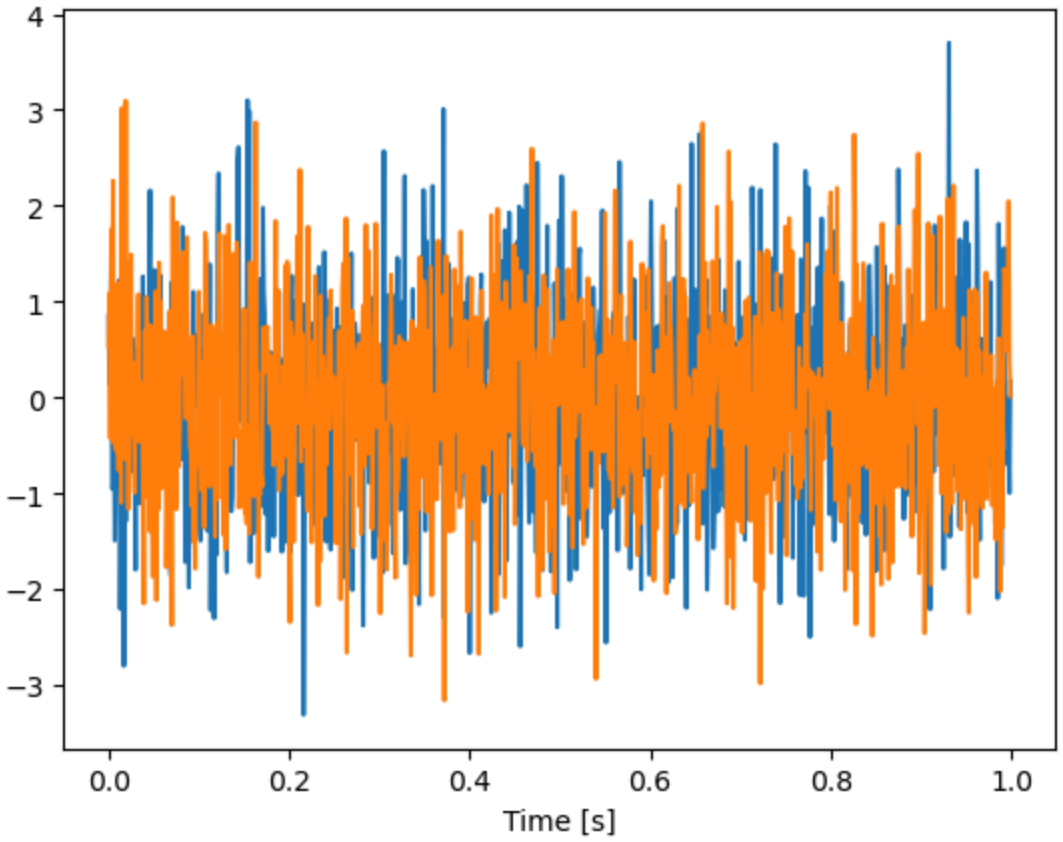


In [1]:
```
# Load modules we'll need.

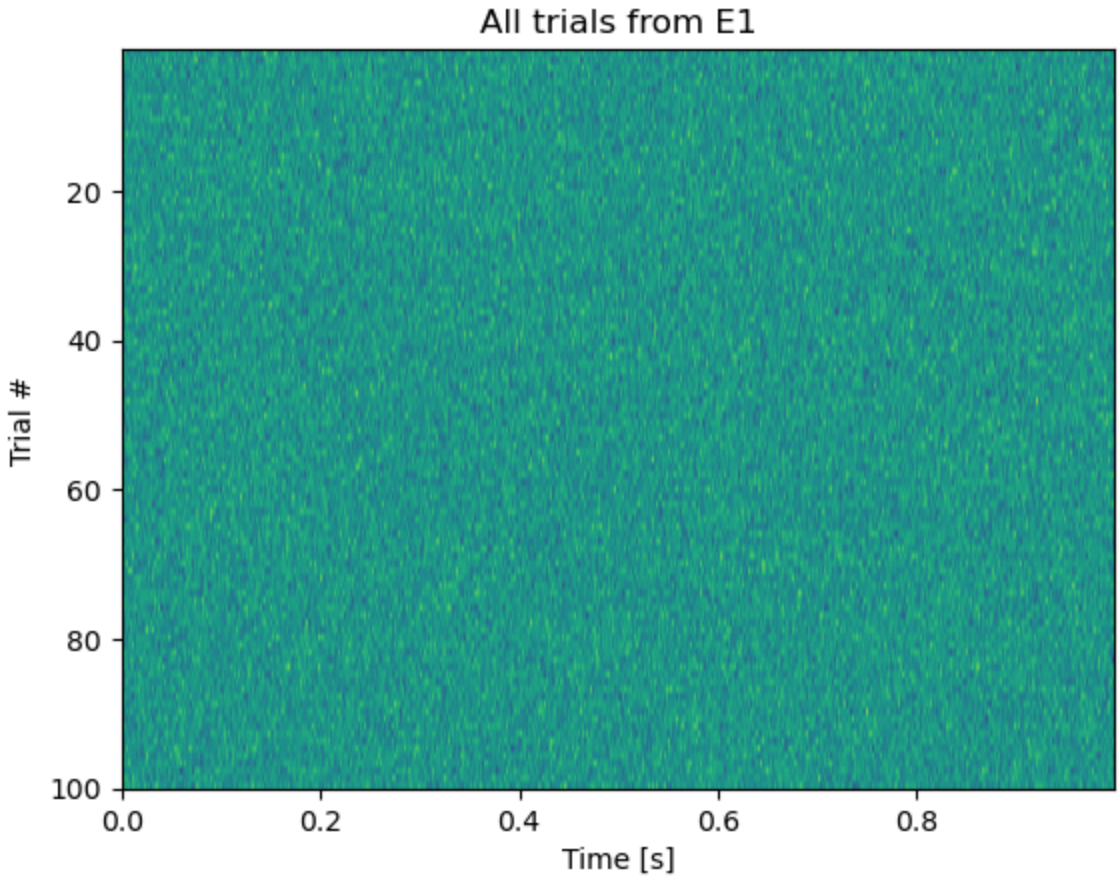
from scipy.io import loadmat
import matplotlib.pyplot as plt
import numpy as np
from scipy.signal import spectrogram
```

In [2]:
```
N = 1000;
K = 100;
dt= 0.001;
T = N*dt;
x = np.random.randn(K,N)
y = np.random.randn(K,N)
t = np.arange(0,N)*dt

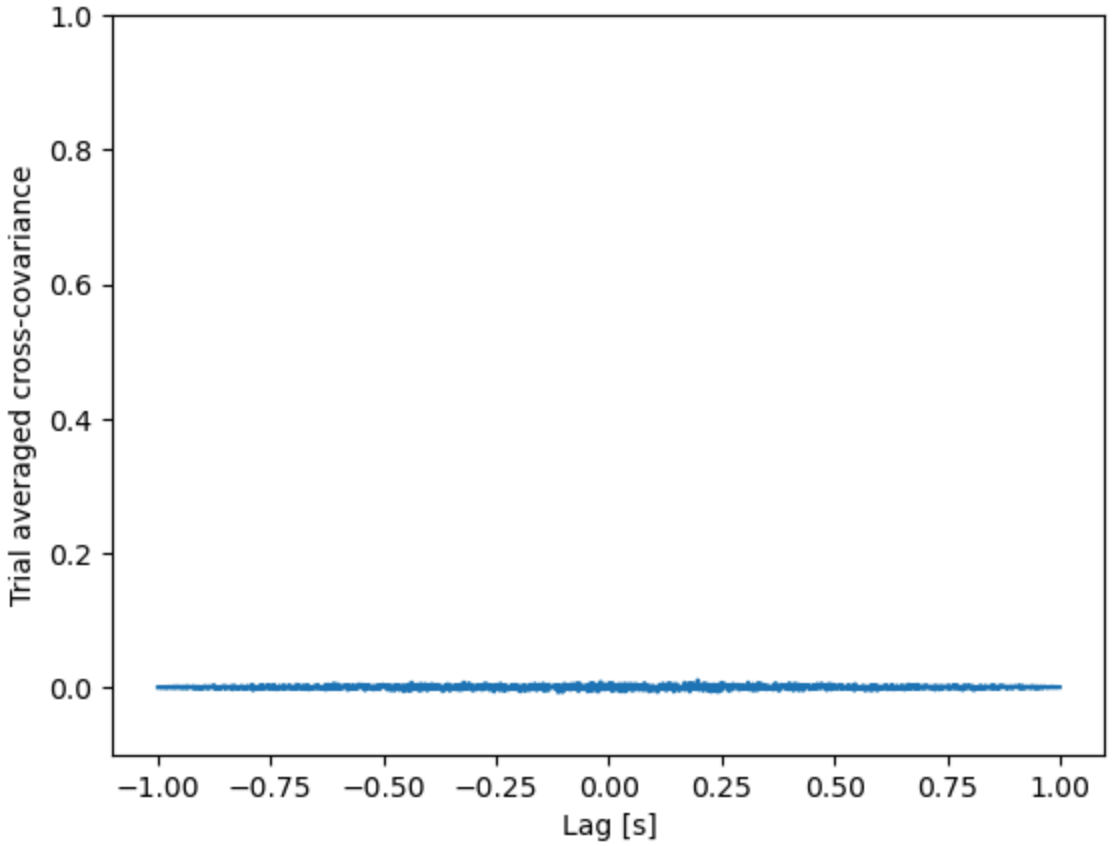
plt.plot(t,x[0,:])
plt.plot(t,y[0,:])
plt.xlabel('Time [s]');
```



In [3]:
```
plt.imshow(x,                                # ... and show the image,
           extent=[min(t), max(t), K, 1],  # ... with meaningful axes,
           aspect='auto')                   # ... and a nice aspect ratio
plt.xlabel('Time [s]')
plt.ylabel('Trial #');
plt.title('All trials from E1');
```



In [5]:
```
cc_xy = 1/N*np.array( [np.correlate(x[k,:], y[k,:], 'full') for k in range(0
cc_xy = np.mean(cc_xy,0)                    # ... average over trials,
lags = np.arange(-N + 1, N)                 # ... create a lag axis,
plt.plot(lags * dt, cc_xy)                  # ... and plot the result.
plt.xlabel('Lag [s]')
plt.ylabel('Trial averaged cross-covariance');
plt.ylim([-0.1, 1]);
```



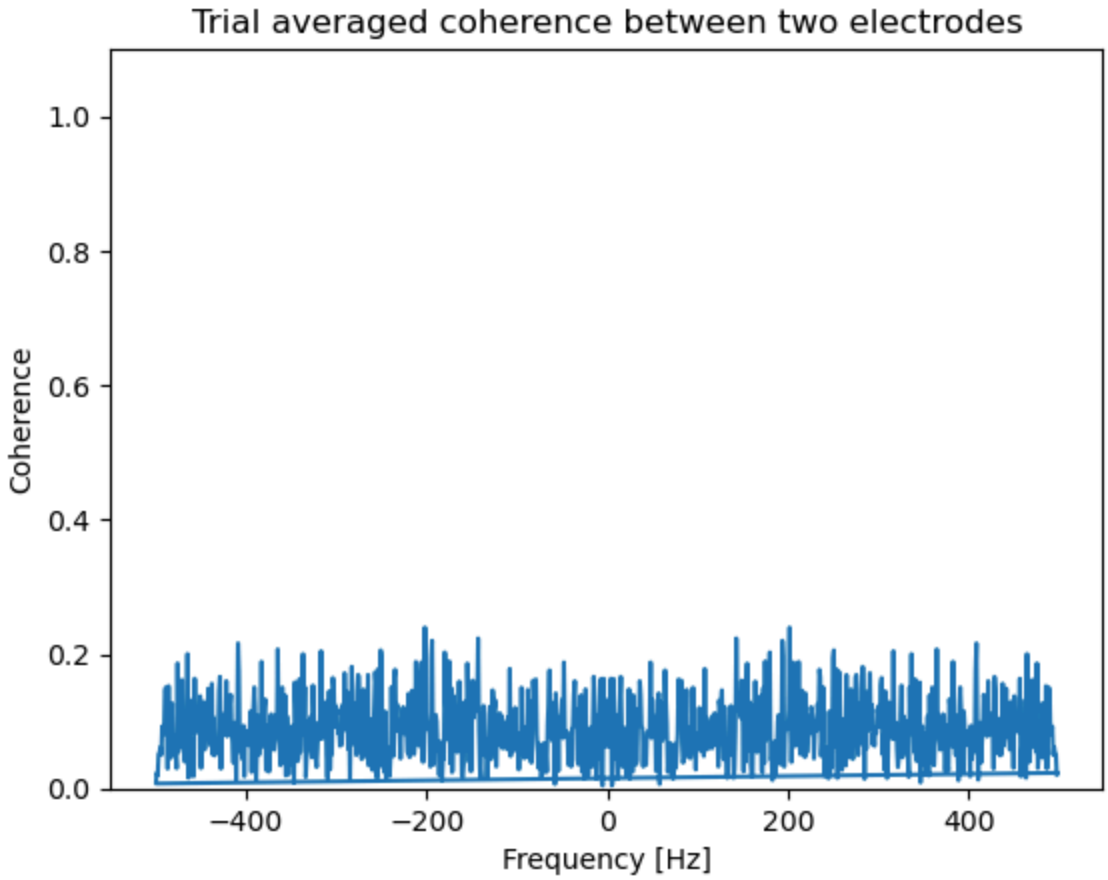
In [6]:
```
# Fourier transforms.
Xf = np.array([np.fft.fft(x0 - np.mean(x0)) for x0 in x])  # Compute Fourier
Yf = np.array([np.fft.fft(y0 - np.mean(y0)) for y0 in y])  # Compute Fourier

# Auto- and cross-spectra.
Sxx = 2*dt**2/T * (Xf * np.conj(Xf))  # Spectrum of x trials
Syy = 2*dt**2/T * (Yf * np.conj(Yf))  # ... and y trials
Sxy = 2*dt**2/T * (Xf * np.conj(Yf))  # ... and the cross spectrum

# Trial average.
Sxx = np.mean(Sxx,0)
Syy = np.mean(Syy,0)
Sxy = np.mean(Sxy,0)

# Calculate coherence.
cohr = np.abs(Sxy) / ( np.sqrt(Sxx) * np.sqrt(Syy) )

f = np.fft.fftfreq(N, dt)                # Define a frequency axis.
plt.plot(f, cohr.real)                   # Plot the coherence.
plt.ylim([0, 1.1])                       # ... with y-axis scaled,
plt.xlabel('Frequency [Hz]')             # ... and with axes labeled.
plt.ylabel('Coherence')
plt.title('Trial averaged coherence between two electrodes');
```


Trial averaged coherence between two electrodes
Coherence
1.0
0.8
0.6
0.4
0.2
0.0
−400
−200
0
200
400
Frequency [Hz]

In [ ]: